

UNIT – 4
ASSESSMENT – 3
PYTHON CODE

NIHARIKA PREEJESH 192524506

```
=====
```

```
# EXPERIMENT 1: DECISION TREE CLASSIFICATION
```

```
#
```

```
=====
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.tree import plot_tree, export_text
```

```
from sklearn.metrics import (
```

```
    accuracy_score,
```

```
    confusion_matrix,
```

```
    classification_report
```

```
)
```

```
# -----
```

```
# STEP 1: LOAD DATASET
```

```
# -----
```

```
iris = load_iris()
```

```
X = pd.DataFrame(  
    iris.data,  
    columns=iris.feature_names  
)
```

```
y = pd.Series(  
    iris.target,  
    name="target"  
)
```

```
# Create complete dataframe
```

```
data = X.copy()
```

```
data["target"] = y
```

```
print("FIRST 5 ROWS")
```

```
print("-----")
```

```
print(data.head())
```

```
# -----
```

```
# STEP 2: DISPLAY DATA TYPES
```

```
# -----
```

```
print("\nDATA TYPES")
```

```
print("-----")
```

```
print(data.dtypes)
```

```
# -----
```

```
# STEP 3: CHECK MISSING VALUES
```

```
# -----
```

```
print("\nMISSING VALUES")
```

```
print("-----")
```

```
print(data.isnull().sum())
```

```
# -----
```

```
# STEP 4: PREPROCESSING
```

```
# -----
```

```
# Iris dataset contains no missing values.  
# Therefore, no missing-value replacement is required.
```

```
# There are no categorical input features,  
# so encoding is not required.
```

```
print("\nPREPROCESSING")  
print("-----")  
print("Missing values: None")  
print("Encoding: Not required")  
print("Dataset is ready for training.")
```

```
# -----  
# STEP 5: TRAIN-TEST SPLIT  
# -----
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.20,  
    random_state=42,
```

```
        stratify=y
    )

    print("\nDATA SPLIT")
    print("-----")
    print("Training samples:", len(X_train))
    print("Testing samples :", len(X_test))

    # -----
    # STEP 6: BUILD DECISION TREE
    # -----

    model = DecisionTreeClassifier(
        criterion="gini",
        random_state=42
    )

    model.fit(X_train, y_train)

    # -----
    # STEP 7: PRINT TREE STRUCTURE
    # -----
```

```
print("\nDECISION TREE STRUCTURE")
```

```
print("-----")
```

```
tree_text = export_text(  
    model,  
    feature_names=list(X.columns)  
)
```

```
print(tree_text)
```

```
# -----
```

```
# STEP 8: IDENTIFY ROOT NODE
```

```
# -----
```

```
root_index = model.tree_.feature[0]
```

```
if root_index != -2:
```

```
    root_feature = X.columns[root_index]
```

```
    print("ROOT NODE")
```

```
    print("-----")
```

```
print(root_feature)
```

```
print(  
    "\nJustification:"  
    "\nThe root node is selected because the"  
    "\nDecision Tree finds this feature to provide"  
    "\nthe best reduction in impurity according"  
    "\nto the Gini Index."  
)
```

```
# -----  
# STEP 9: PREDICTION  
# -----
```

```
y_pred = model.predict(X_test)
```

```
# -----  
# STEP 10: ACCURACY  
# -----
```

```
accuracy = accuracy_score(  
    y_test,
```

```

        y_pred
    )

print("\nACCURACY")
print("-----")
print("Accuracy =", round(accuracy, 4))

# -----
# STEP 11: CONFUSION MATRIX
# -----

cm = confusion_matrix(
    y_test,
    y_pred
)

print("\nCONFUSION MATRIX")
print("-----")
print(cm)

# -----
# STEP 12: CLASSIFICATION REPORT

```



```
# -----

print("\nCLASSIFICATION REPORT")
print("-----")

print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names
    )
)

# -----
# STEP 13: FIND MISCLASSIFIED INSTANCES
# -----

print("\nMISCLASSIFIED INSTANCES")
print("-----")

misclassified = []
```

```
for i in range(len(y_test)):

    if y_test.iloc[i] != y_pred[i]:

        misclassified.append(
            (
                X_test.index[i],
                y_test.iloc[i],
                y_pred[i]
            )
        )

if len(misclassified) == 0:

    print("No misclassified instances.")

else:

    for item in misclassified[:5]:

        index, actual, predicted = item
```

```
print(
    "Index:", index,
    "| Actual:", iris.target_names[actual],
    "| Predicted:", iris.target_names[predicted]
)
```

```
# -----
```

```
# STEP 14: FEATURE IMPORTANCE
```

```
# -----
```

```
print("\nFEATURE IMPORTANCE")
```

```
print("-----")
```

```
importance = pd.Series(
    model.feature_importances_,
    index=X.columns
).sort_values(
    ascending=False
)
```

```
print(importance)
```

```
# -----
```

```
# STEP 15: PLOT TREE
```

```
# -----
```

```
plt.figure(figsize=(16, 10))
```

```
plot_tree(  
    model,  
    feature_names=X.columns,  
    class_names=iris.target_names,  
    filled=True  
)
```

```
plt.title("Decision Tree - Iris Dataset")
```

```
plt.show()
```

```
=====
```

```
# EXPERIMENT 2: Q-LEARNING
```

```
# 4 x 4 GRID WORLD
```

```
=====
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# -----
```

```
# STEP 1: GRID WORLD
```

```
# -----
```

```
ROWS = 4
```

```
COLS = 4
```

```
START = 0
```

```
GOAL = 15
```

```
OBSTACLE = 6
```

```
# Number of states
```

```
NUM_STATES = ROWS * COLS
```

```
# Actions
```

```
ACTIONS = {
```

```
    0: "UP",
```

```
    1: "DOWN",
```

```
    2: "LEFT",
```

```
    3: "RIGHT"
```

```
}
```

```
NUM_ACTIONS = 4
```

```
# -----
```

```
# STEP 2: HYPERPARAMETERS
```

```
# -----
```

```
alpha = 0.1
```

```
gamma = 0.9
```

```
epsilon = 0.2
```

```
episodes = 100
```

```
max_steps = 100
```

```
print("Q-LEARNING PARAMETERS")
```

```
print("-----")
print("Learning Rate (alpha) :", alpha)
print("Discount Factor (gamma):", gamma)
print("Exploration Rate (epsilon):", epsilon)
print("Episodes:", episodes)
```

```
# -----
# STEP 3: INITIALIZE Q-TABLE
# -----
```

```
Q = np.zeros(
    (NUM_STATES, NUM_ACTIONS)
)
```

```
# -----
# STEP 4: CONVERT STATE TO ROW/COLUMN
# -----
```

```
def position(state):
```

```
    row = state // COLS
```

```
    col = state % COLS
```

```
return row, col
```

```
# -----
```

```
# STEP 5: GET NEXT STATE
```

```
# -----
```

```
def get_next_state(state, action):
```

```
    row, col = position(state)
```

```
    new_row = row
```

```
    new_col = col
```

```
    if action == 0:    # UP
```

```
        new_row -= 1
```

```
    elif action == 1:  # DOWN
```

```
        new_row += 1
```

```
    elif action == 2:  # LEFT
```



```
new_col -= 1
```

```
elif action == 3:    # RIGHT
```

```
new_col += 1
```

```
# Boundary condition
```

```
if(
```

```
new_row < 0 or
```

```
new_row >= ROWS or
```

```
new_col < 0 or
```

```
new_col >= COLS
```

```
):
```

```
return state, -1
```

```
next_state = (
```

```
new_row * COLS + new_col
```

```
)
```

```
# Goal
```

```
if next_state == GOAL:
```

```
return next_state, 10
```

```
# Obstacle
if next_state == OBSTACLE:
    return next_state, -5

# Normal movement
return next_state, -1

# -----
# STEP 6: EPSILON-GREEDY ACTION SELECTION
# -----

def choose_action(state):

    # Exploration
    if np.random.random() < epsilon:

        return np.random.randint(
            NUM_ACTIONS
        )

    # Exploitation
```

```
return np.argmax(  
    Q[state]  
)
```

```
# -----  
# STEP 7: Q-LEARNING  
# -----
```

```
np.random.seed(42)
```

```
cumulative_rewards = []
```

```
# Representative states
```

```
state1 = 0
```

```
state2 = 5
```

```
saved_q_values = {}
```

```
for episode in range(1, episodes + 1):
```

```
    state = START
```

```
total_reward = 0
```

```
for step in range(max_steps):
```

```
    # Select action
```

```
    action = choose_action(state)
```

```
    # Take action
```

```
    next_state, reward = get_next_state(  
        state,  
        action  
    )
```

```
    # Current Q value
```

```
    old_q = Q[state, action]
```

```
    # Best future Q value
```

```
    best_future_q = np.max(  
        Q[next_state]  
    )
```

```
    # Q-Learning update
```

```
new_q = old_q + alpha * (  
    reward  
    + gamma * best_future_q  
    - old_q  
)
```

```
Q[state, action] = new_q
```

```
total_reward += reward
```

```
state = next_state
```

```
# Stop if goal reached
```

```
if state == GOAL:
```

```
    break
```

```
cumulative_rewards.append(  
    total_reward  
)
```

```
# -----
```

```
# SAVE Q VALUES
```

```
# -----
```

```
if episode in [1, 50, 100]:
```

```
    saved_q_values[episode] = {  
        "State 0": Q[state1].copy(),  
        "State 5": Q[state2].copy()  
    }
```

```
# -----
```

```
# STEP 8: DISPLAY Q VALUES
```

```
# -----
```

```
print("\nQ-TABLE EVOLUTION")
```

```
print("-----")
```

```
for episode in [1, 50, 100]:
```

```
    print("\nEpisode", episode)
```

```
    print(
```

```
"State 0:",  
np.round(  
    saved_q_values[episode]["State 0"],  
    3  
)  
)
```

```
print(  
    "State 5:",  
    np.round(  
        saved_q_values[episode]["State 5"],  
        3  
    )  
)
```

```
print(  
    "Actions:",  
    ["UP", "DOWN", "LEFT", "RIGHT"]  
)
```

```
# -----
```

```
# STEP 9: FINAL Q TABLE
```

```
# -----
```

```
print("\nFINAL Q-TABLE")
```

```
print("-----")
```

```
print(
```

```
    np.round(Q, 3)
```

```
)
```

```
# -----
```

```
# STEP 10: FINAL POLICY
```

```
# -----
```

```
print("\nFINAL LEARNED POLICY")
```

```
print("-----")
```

```
symbols = {
```

```
    0: "↑",
```

```
    1: "↓",
```

```
    2: "←",
```



```
3: "→"  
}
```

```
for row in range(ROWS):
```

```
    line = ""
```

```
    for col in range(COLS):
```

```
        state = row * COLS + col
```

```
        if state == GOAL:
```

```
            line += " G "
```

```
        elif state == OBSTACLE:
```

```
            line += " X "
```

```
        else:
```

```
            best_action = np.argmax(
```

```
        Q[state]
    )
```

```
    line += (
        " "
        + symbols[best_action]
        + " "
    )
```

```
print(line)
```

```
# -----
```

```
# STEP 11: CUMULATIVE REWARD PLOT
```

```
# -----
```

```
plt.figure(figsize=(10, 5))
```

```
plt.plot(
    range(1, episodes + 1),
    cumulative_rewards
)
```

```
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")

plt.title(
    "Q-Learning Cumulative Reward"
)

plt.grid(True)

plt.show()

# -----
# STEP 12: AVERAGE REWARD
# -----

print("\nCONVERGENCE ANALYSIS")
print("-----")

first_10 = np.mean(
    cumulative_rewards[:10]
```

)

```
last_10 = np.mean(  
    cumulative_rewards[-10:]  
)
```

```
print(  
    "Average reward Episodes 1-10 :",  
    round(first_10, 2)  
)
```

```
print(  
    "Average reward Episodes 91-100:",  
    round(last_10, 2)  
)
```

```
if last_10 > first_10:
```

```
    print(  
        "The agent shows learning improvement."  
    )
```

else:

print(

"The agent has not shown significant improvement."

)